

DevPath

Learn by Building. Ship Something Real.

Database Design Document · Entity Relationship Model · v1.0

Prepared by **Darshan Jain**, **Francis Reuben R** and **Harshith B A**

March 2026

1. Title

Project Name: DevPath - Project-Driven Programming Learning Platform

Document Type: Database Design Specification - Conceptual ER Model

Scope: This document covers the conceptual database design for the DevPath platform, including entity identification, attribute classification, relationship mapping, cardinality constraints, participation constraints, and the complete Entity-Relationship (ER) Diagram following the Chen notation standard.

Version: 1.0 · **Status:** Draft

2. Abstract

DevPath is an AI-augmented, project-driven learning platform that unifies structured programming education with professional product development tooling. It targets beginner and intermediate developers by treating the learning path and the product lifecycle as a single co-dependent workflow. The platform is built on four pillars: a learn-by-building model where every lesson maps to a concrete project deliverable; a dual-mode workspace (Learn Mode and Build Mode) with bidirectional progress linkage; role-based collaborative team workspaces for groups of up to four learners; and a skill-gated Kanban board that enforces lesson completion as a prerequisite to task progression at the data layer. An AI subsystem fulfils four roles across these pillars: learning path generation, Kanban board population, mentor-coach interaction, and automated code evaluation with structured feedback. This document presents the conceptual database design for DevPath using Chen ER Model notation, producing a schema of seven entity sets, nine relationship sets, and a full constraint specification - serving as the formal input to the subsequent logical and physical design phases.

2.1 Platform Summary

DevPath serves two primary user profiles - beginner programmers and intermediate developers - operating across three subscription tiers: Starter (free), Pro (₹529/month), and Team (₹1249/month per team). The platform's key differentiators are:

- AI-generated, project-scoped learning paths that replace generic curricula
- A skill-gated Kanban board where task cards are locked until prerequisite lessons are completed
- Role-based team workspaces where 2–4 learners collaborate on a shared product as a simulated dev team
- Automated AI code evaluation that reviews submissions, provides structured feedback, and unlocks the next task card
- A "Ship It" milestone system that replaces course certificates with portfolio-ready product case studies

2.2 Objectives of the Database Design

The database must support the following operational requirements:

- Store and retrieve user profiles, subscription tier, and role assignments
- Track project ownership, project structure, and completion state across individual and team workspaces
- Manage course catalogue, lessons, and skill-gate dependencies between lessons and Kanban tasks
- Record team composition, member roles, and team-level project associations
- Persist code submission history, AI evaluation scores, and feedback per user per lesson
- Support blueprint generation metadata and AI-generated tech stack recommendations per project
- Enforce referential integrity across all cross-entity associations to maintain platform data consistency

2.3 Design Methodology

The conceptual design follows the Chen ER Model notation (Peter Pin-Shan Chen, 1976) as taught in the RV University DBMS curriculum. Entities are classified as strong or weak; attributes are classified as simple, composite, multivalued, derived, or key attributes. Relationships are characterised by degree, cardinality ratio, and participation constraints (total vs. partial). The resulting ER Diagram is presented in Section 4.

3. Entity and Relationship Descriptions

3.1 Entity Summary

The DevPath database comprises 7 entities. Two are weak entities (KANBAN_TASK and SUBMISSION) that cannot be uniquely identified without their respective owner entities.

Entity	Type	Key Attributes	Purpose
USER	Strong	user_id (PK), email, name, role	Represents all registered learners and team members
PROJECT	Strong	project_id (PK), proj_name, completion%*	A product being built by a user or team
COURSE	Strong	course_id (PK), title, tech_stack	A structured programming learning track
LESSON	Strong	lesson_id (PK), content, tags**	An individual learning unit within a course
KANBAN_TASK	Weak	task_name† + project_id (FK)	A task card within a project's Kanban board
TEAM	Strong	team_id (PK), team_name, roles**	A collaborative group of learners (up to 4 members)
BLUEPRINT	Strong	blueprint_id (PK), tech_recgs, ai_gen*	AI-generated project blueprint and tech recommendations
SUBMISSION	Weak	sub_date† + user_id (FK)	A code submission by a user for evaluation

* Derived attribute ** Multivalued attribute † Partial key (discriminator)

3.2 Relationship Summary

The following relationships model the associations between entities on the DevPath platform. Cardinality and participation constraints are noted for each.

Relationship	Entities	Cardinality	Participation & Notes
ENROLLS_IN	USER ↔ COURSE	M : N	Total on User side; has descriptive attr: grade
OWNS	USER ↔ PROJECT	1 : N	Total on User side; each project has one owner
CONTAINS	PROJECT ↔ KANBAN_TASK	1 : N	Total on both sides; identifying relationship (double diamond)

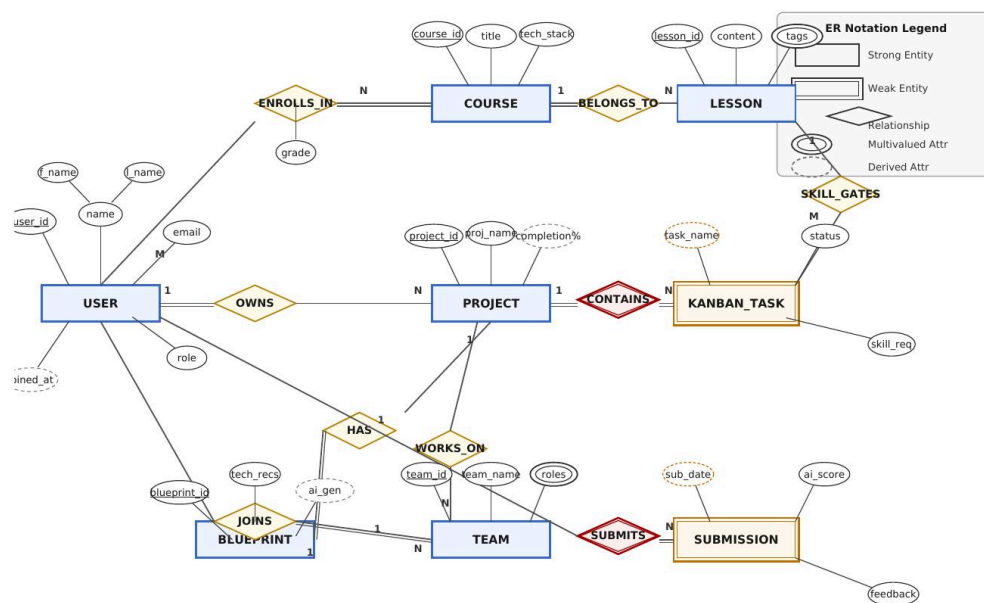
BELONGS_TO	COURSE ↔ LESSON	1 : N	Total on Lesson side; lessons cannot exist without a course
SKILL_GATES	KANBAN_TASK ↔ LESSON	M : 1	A task is unlocked only after its gating lesson is completed
JOINS	USER ↔ TEAM	M : N	Users may belong to multiple teams; teams may have multiple users
WORKS_ON	TEAM ↔ PROJECT	N : 1	A team works on one project; a project may have one team
HAS	PROJECT ↔ BLUEPRINT	1 : 1	Total on Project side; every project has exactly one blueprint
SUBMITS	USER ↔ SUBMISSION	1 : N	Total on Submission side; identifying relationship (double diamond)

3.3 Key Integrity Constraints

Constraint Type	Entity / Attribute	Rule
Entity Integrity	All entities	Primary key attributes cannot be NULL
Referential Integrity	KANBAN_TASK.project_id	Must reference PROJECT.project_id (ON DELETE CASCADE)
Referential Integrity	SUBMISSION.user_id	Must reference USER.user_id (ON DELETE CASCADE)
Referential Integrity	LESSON.course_id (FK)	Must reference COURSE.course_id (ON DELETE CASCADE)
Domain Constraint	USER.role	CHECK (role IN ('learner', 'mentor', 'admin'))
Domain Constraint	KANBAN_TASK.status	CHECK (status IN ('locked', 'in_progress', 'done'))
Domain Constraint	SUBMISSION.ai_score	CHECK (ai_score BETWEEN 0 AND 100)
Key Constraint	USER.email	UNIQUE - no two users share an email address

4. Entity Relationship Diagram

The following ER Diagram represents the complete conceptual schema for the DevPath platform. It uses the Chen notation standard as specified in the DBMS Unit 2 curriculum. Strong entities are drawn with a single-border rectangle; weak entities with a double-border rectangle. Relationships are drawn as diamonds; identifying relationships (connecting weak entities to their owners) use a double diamond. Attributes use the following notation: single oval (simple), oval with sub-ovals (composite), double oval (multivalued), dashed oval (derived), and underlined attribute name (key/partial key).



DevPath — Entity Relationship Diagram

Double border = Weak Entity / Identifying Relationship | Double line = Total Participation | Dashed oval = Derived Attr | Double oval = Multivalued Attr

Figure 1: DevPath ER Diagram (Chen Notation)

5. ER-to-Table Reduction Notes

The following summarises how the ER schema will reduce to a relational schema in the logical design phase, following the standard 7-step algorithm (Elmasri & Navathe):

5.1 Strong Entities → Tables

Each strong entity (USER, PROJECT, COURSE, LESSON, TEAM, BLUEPRINT) will produce one table. Composite attributes (name → f_name, l_name) will be expanded into component columns. Derived attributes (completion%, ai_gen) will be omitted from the schema and computed at query time.

5.2 Weak Entities → Tables with Composite PK

KANBAN_TASK will produce a table with PK = (project_id, task_name). SUBMISSION will produce a table with PK = (user_id, sub_date). Both tables carry a foreign key referencing their owner entity with ON DELETE CASCADE.

5.3 Multivalued Attributes → Separate Tables

LESSON.tags → LESSON_TAG(lesson_id, tag) with PK = (lesson_id, tag). TEAM.roles → TEAM_ROLE(team_id, role) with PK = (team_id, role).

5.4 M:N Relationships → Junction Tables

ENROLLS_IN → ENROLLMENT(user_id, course_id, grade) with PK = (user_id, course_id). JOINS → TEAM_MEMBER(user_id, team_id) with PK = (user_id, team_id).

5.5 1:N Relationships → FK Migration

OWNS: project_id table receives user_id as FK. BELONGS_TO: LESSON table receives course_id as FK. WORKS_ON: PROJECT table receives team_id as FK. SKILL_GATES: KANBAN_TASK table receives lesson_id as FK. SUBMITS: Already handled by weak entity reduction.

5.6 1:1 Relationship

HAS (PROJECT ↔ BLUEPRINT): PROJECT table receives blueprint_id as FK (since PROJECT has total participation - every project must have a blueprint). This minimises NULLs.

End of Document

DevPath DBMS Design v1.0 · March 2026 · For Review